Adapter

er · Wrap legacy XML SOAP payment gateway as PaymentProcessor · Build with V

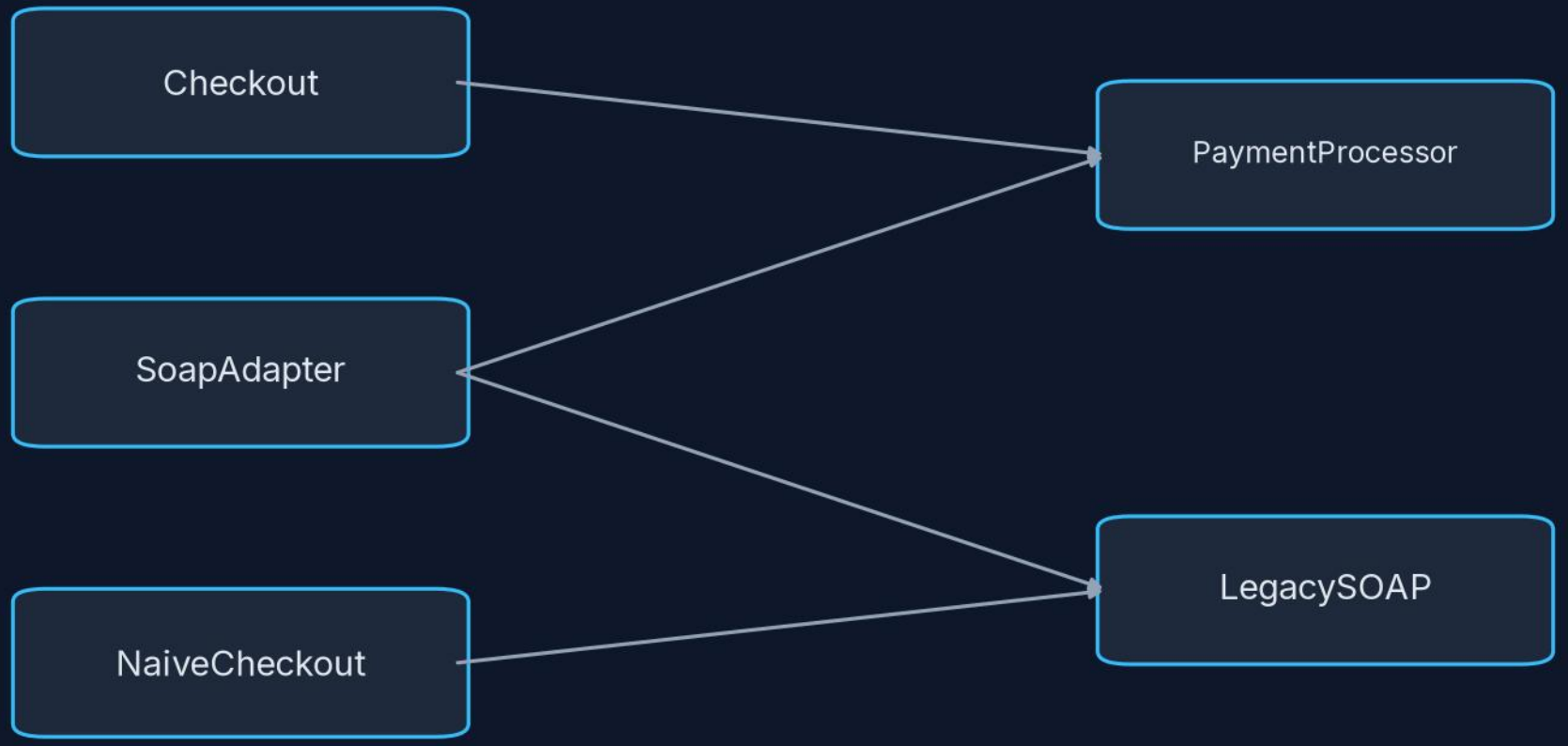
Today's Hook

- Today's force: Wrap legacy XML SOAP payment gateway as PaymentProcessor
- We earn Adapter by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: SoapPaymentAdapter

The Problem

- Checkout must call PaymentProcessor.charge(customer, cents, currency).
- LegacySoapGateway only accepts XML envelopes and returns <Txn>/<Fault>.
- NaiveCheckout builds XML and scrapes ack in every controller.
- Success: OK:SOAP-... for USD; FAILED for non-positive; INR rejected.

Structure



Adapter: Wrap legacy XML SOAP payment gateway as PaymentPro

Real-World Scenario

- Running example: Wrap legacy XML SOAP payment gateway as PaymentProcessor
- Key types: SoapPaymentAdapter
- Happy-path stdout cue: OK:SOAP-alice-1998
- Failure-path stdout cue: FAILED:<Fault>non-positive</Fault>

Naive Code

```java
package com.buildwithwaqar.patterns.gof.structural.adapter.problem;

/**
 * Naive: checkout knows SOAP XML and dollar conversion.
 * Every call site duplicates envelope formatting and response scraping.
 */
public final class NaiveCheckout {
    private final LegacySoapGateway gateway = new LegacySoapGateway();

    public String pay(String customerId, int amountCents) {
        double dollars = amountCents / 100.0;
        String xml = "<Envelope><Customer>" + customerId + "</Customer><Amount>"
                + dollars + "</Amount></Envelope>";
        String ack = gateway.submitXml(xml);
        if (ack.contains("<Fault>")) {
            return "FAILED:" + ack;
        }
        int ts = ack.indexOf("<Txn>") + 5;
        int te = ack.indexOf("</Txn>");
        return "OK:" + ack.substring(ts, te);
    }
}
```

Why Naive Hurts

- Naive: checkout knows SOAP XML and dollar conversion. Every call site duplicates envelope formatting and response scraping.
- Without Adapter, call sites absorb the wrong details.
- Failure mode to show on camera: FAILED:<Fault>non-positive</Fault>
- New requirements force shotgun edits across the naive code.

Solution

```
package com.buildwithwaqar.patterns.gof.structural.adapter.problem;

/** Adapter: translates PaymentProcessor calls into LegacySoapGateway XML. */
public final class SoapPaymentAdapter implements PaymentProcessor {
    private final LegacySoapGateway gateway;

    public SoapPaymentAdapter(LegacySoapGateway gateway) {
        this.gateway = gateway;
    }

    @Override
    public PaymentResult charge(String customerId, int amountCents, String currency) {
        if (!"USD".equalsIgnoreCase(currency)) {
            return new PaymentResult(false, null, "unsupported currency " + currency);
        }
        double dollars = amountCents / 100.0;
        String xml = "<Envelope><Customer>" + customerId
                + "</Customer><Amount>" + dollars + "</Amount><Currency>USD</Currency></Envelope>";
        String ack = gateway.submitXml(xml);
        if (ack.contains("<Fault>")) {
            return new PaymentResult(false, null, ack);
        }
```

Solution (2)

```
        int ts = ack.indexOf("<Txn>") + 5;
        int te = ack.indexOf("</Txn>");
        return new PaymentResult(true, ack.substring(ts, te), "charged");
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.structural.adapter.problem;

/** Problem: checkout must talk PaymentProcessor; legacy SOAP gateway speaks XML. */
public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Adapter problem: legacy SOAP payment gateway ===");
        System.out.println("--- naive (controller knows SOAP XML) ---");
        System.out.println(new NaiveCheckout().pay("alice", 1999));
        System.out.println("--- adapter (PaymentProcessor) ---");
        var checkout = new CheckoutService(new SoapPaymentAdapter(new LegacySoapGateway()));
        System.out.println(checkout.checkout("alice", 1999));
        System.out.println(checkout.checkout("alice", 0));
        System.out.println(new SoapPaymentAdapter(new LegacySoapGateway()).charge("a", 100, "INR"));
    }
}
```

Walkthrough Output

```
=== Adapter problem: legacy SOAP payment gateway ===
--- naive (controller knows SOAP XML) ---
OK:SOAP-alice-1998
--- adapter (PaymentProcessor) ---
OK:SOAP-alice-1998
FAILED:<Fault>non-positive</Fault>
PaymentResult[ok=false, transactionId=null, message=unsupported currency INR]
```

Step-by-Step Trace

- Match stdout to stages — look for: OK:SOAP-alice-1998
- Then the injected failure: FAILED:<Fault>non-positive</Fault>
- Re-run: java -cp ... 08-adapter ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for Adapter
// Prefer records + sealed where they shrink the demo.
// Problem under test: Wrap legacy XML SOAP payment gateway as PaymentProcessor
```

Trade-offs

- Use Adapter when «Wrap legacy XML SOAP payment gateway as PaymentProcessor» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Adapter in product language.

Related Patterns

- Name the pattern usually confused with Adapter and separate the forces.
- Sketch the roles for «Wrap legacy XML SOAP payment gateway as PaymentProcessor» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Wrap legacy XML SOAP payment gateway as PaymentProcessor
- Naive pain → Adapter boundary → Demo stdout.
- Remember the failure cue: FAILED:<Fault>non-positive</Fault>
- Break a ProblemTest assertion, watch it fail, restore it.